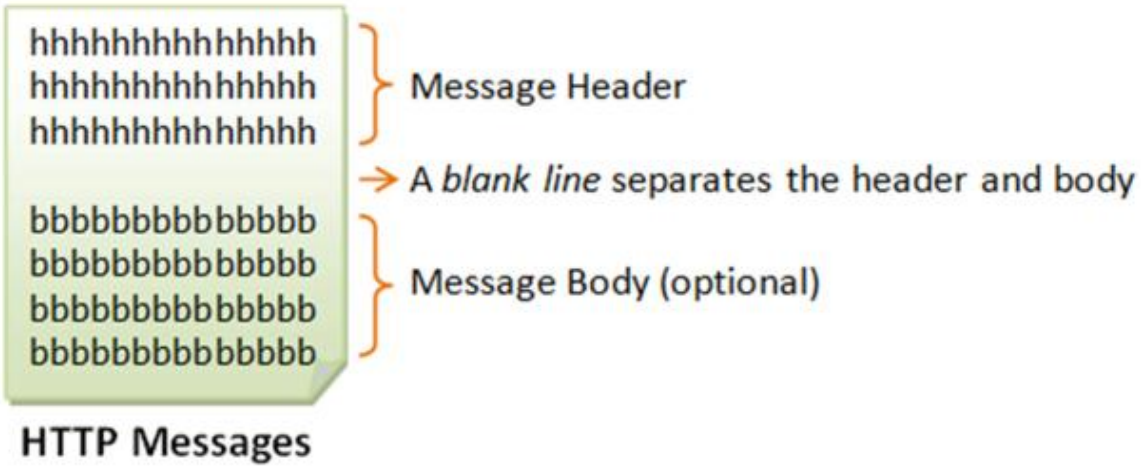
🌐 Basic Web Concepts Covered

1. HTTP (HyperText Transfer Protocol)

HTTP is a fundamental protocol for communication between a client (like a web browser) and a server.

- **Communication:** HTTP clients and servers communicate by sending **text messages**.
    - The **client** sends a **request message** to the server.
    - The **server** returns a **response message**.
- **Message Structure:** An HTTP message consists of a **message header** and an **optional message body**, which are separated by a **blank line**.

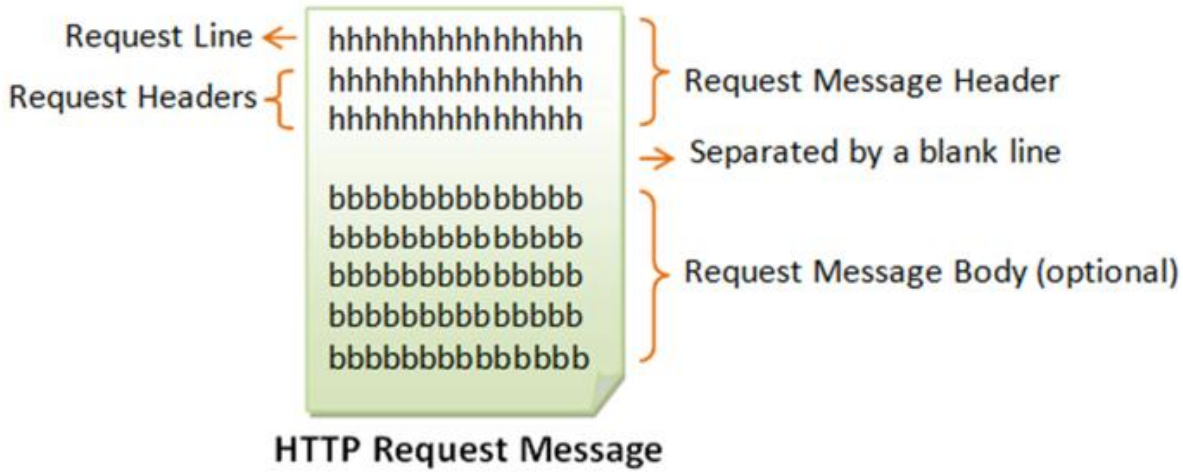


---

2. HTTP Request Message

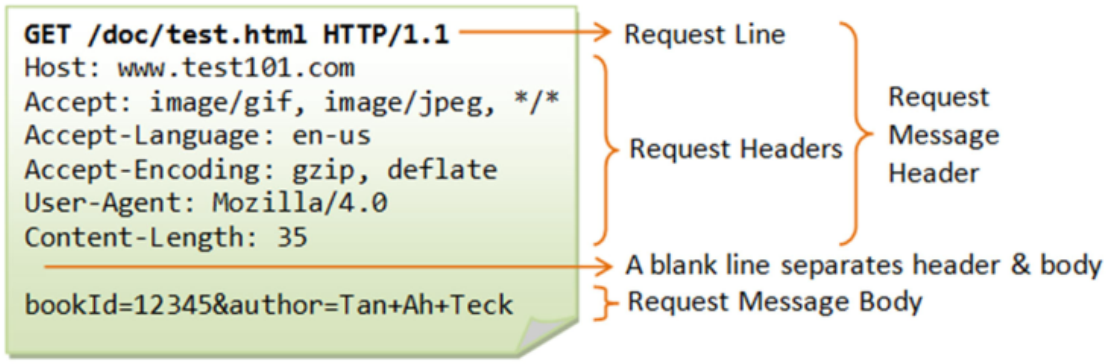
The client sends a request message to the server

Request Message Part	Description	Structure/Example
Request Line	The first line of the header, which is mandatory.	Request method name/request URI HTTP version
Request Headers	Optional lines of information in the form of **name: value** pairs. They can specify details like the host, accepted content types, or language.	Host: www.xyz.com, Accept: image/gif, image/jpeg, */*
Blank Line	Separates the header from the body.	
Request Body	**Optional**; used to carry data like URL-encoded query strings for a POST request or uploaded files.	bookId=12345&author=Tan+Ah+Teck



Example

The following shows a sample HTTP request message:



HTTP Request Methods 🚀

The HTTP protocol defines a set of request methods a client can use:

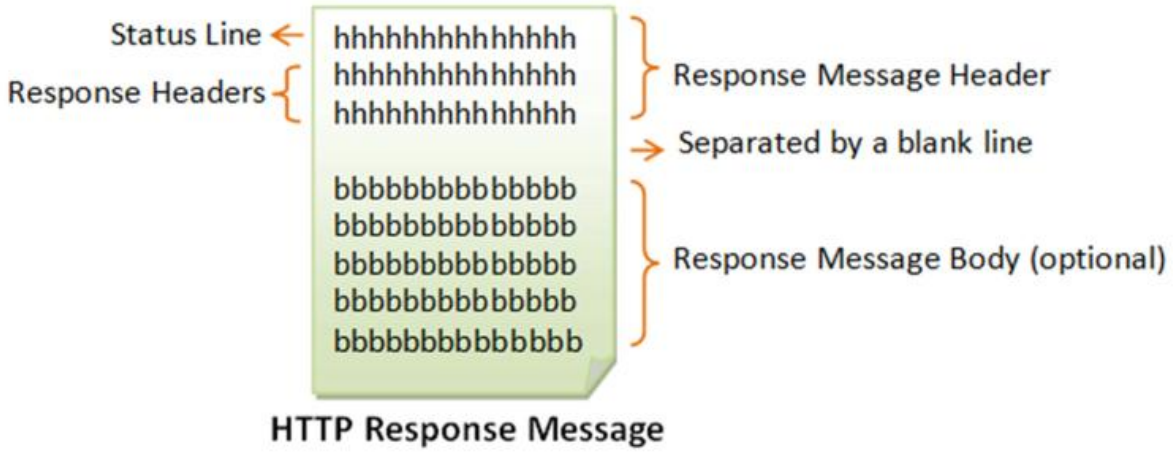
- **GET:** The most common method; used to **request a web resource** (document) from the server.
- **POST:** Used to **post data up to the web server** (e.g., submitting HTML form data or uploading a file). The data is sent in the **request body**.
- **HEAD:** Requests only the **header** that a GET request would have obtained, useful for checking if a resource has been modified against a local cache copy.
- **PUT:** Asks the server to **store the data**.
- **DELETE:** Asks the server to **delete the data**.
- **OPTIONS:** Asks the server to return the **list of request methods it supports**.
- **TRACE:** Asks the server to return a **diagnostic trace** of the actions it takes (sends a copy of the entire request message back)

---

3. HTTP Response Message

The server sends a response message back to the client.

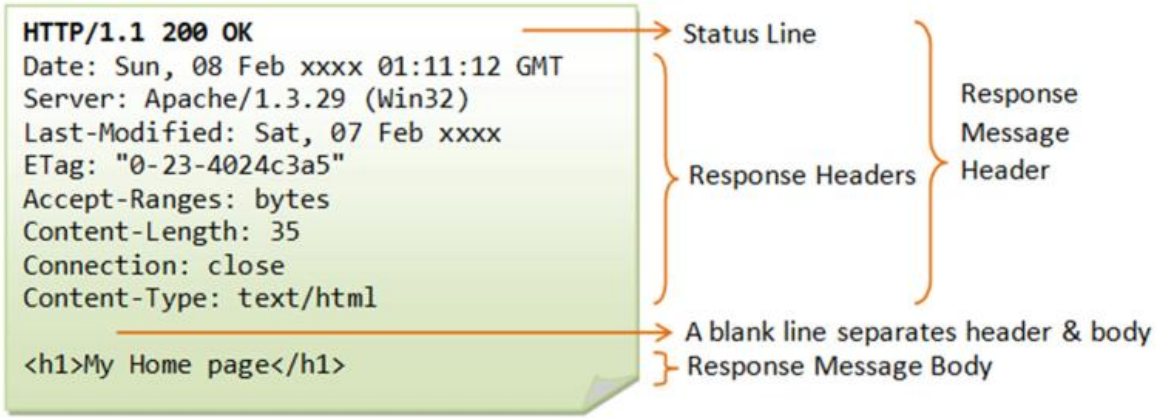
Response Message Part	Description	Structure/Example
Status Line	The first line of the response header.	HTTP/version status-code reason-phrase
Response Headers	Optional lines of information like Content-Type or Content-Length.	Content-Type: text/html
Blank Line	Separates the header from the body.	
Response Body	**Optional**; contains the **resource data requested** (e.g., an HTML file).	My Home page



HTTP Response Message

Example

The following shows a sample response message:



Response Status Codes ✅

The **status code** is a 3-digit number in the status line that indicates the **outcome of the request**

Status Code Range	Category	Description
1xx	Informational	Request received, server is continuing the process.
2xx	**Success**	The request was successfully received, understood, accepted, and serviced (e.g., **200 OK**).
3xx	Redirection	Further action must be taken to complete the request (e.g., **304 Not Modified**).
4xx	**Client Error**	The request contains bad syntax or cannot be understood (e.g., **400 Bad Request**, **404 Not Found**).
5xx	**Server Error**	The server failed to fulfill a valid request (e.g., **500 Internal Server Error**).

Some commonly encountered status codes are:

- 100 Continue: The server received the request and in the process of giving the response.
- 200 OK: The request is fulfilled.
- 304 Not Modified: In response to the If-Modified-Since conditional GET request, the server notifies that the resource requested has not been modified.
- 400 Bad Request: Server could not interpret or understand the request, probably syntax error in the request message.
- 401 Authentication Required: The requested resource is protected and require client's credential (username/password). The client should re-submit the request with his credential (username/password).
- 403 Forbidden: Server refuses to supply the resource, regardless of identity of client.
- 404 Not Found: The requested resource cannot be found in the server.
- 405 Method Not Allowed: The request method used, e.g., POST, PUT, DELETE, is a valid method. However, the server does not allow that method for the resource requested.
- 408 Request Timeout:
- 414 Request URI too Large:
- 501 Method Not Implemented: The request method used is invalid (could be caused by a typing error, e.g., "GET" misspell as "Get").
- 503 Service Unavailable: Server cannot response due to overloading or maintenance. The client can try again later.

---

4. HTTP Connection Behavior (HTTP/1.0 vs. HTTP/1.1)

- **HTTP/1.0:** By default, the server **closes the TCP connection** after the response is delivered. A client can request a persistent connection using the optional header **Connection: Keep-Alive**.
- **HTTP/1.1:** Uses **persistent (keep-alive) connection as default**.

---

5. Testing HTTP Requests

You can test HTTP requests using a utility program like **"telnet"** to establish a TCP connection and issue **raw HTTP requests**. Alternatively, you can write your own network program (like the Java examples provided in the lecture) to issue raw requests.

```
import java.net.*;
import java.io.*;
public class HttpClient {
  public static void main(String[] args) throws IOException {
    // The host and port to be connected.
    String host = "127.0.0.1";
    int port = 8000;
    // Create a TCP socket and connect to the host:port.
    Socket socket = new Socket(host, port);
    // Create the input and output streams for the network socket.
    BufferedReader in
      = new BufferedReader(
          new InputStreamReader(socket.getInputStream()));
    PrintWriter out
      = new PrintWriter(socket.getOutputStream());
```

```
    // Send request to the HTTP server.
    out.println("GET /index.html HTTP/1.0");
    // blank line separating header & body
    out.println();
    out.flush();
    // Read the response and display on console.
    String line;
    // readLine() returns null if server close the network socket.
    while((line = in.readLine()) != null) {
      System.out.println(line);
    }
    // Close the I/O streams.
    in.close();
    out.close();
  }
}
```

Java Http Server and Java Http Client

- **We are trying to write our own java http client and a java http server.**
- **our main aim is to send an HTML document in the body of http and in return get a sequence response messages from our own http server.**
- **The code for both client and server is as follows:**

```
public class Client {
public static void main(String[] args) {
try {
URL url = new URL("http://localhost:12001");
HttpURLConnection con = (HttpURLConnection) url.openConnection();
//If its argument is set to true, then output is allowed
con.setDoOutput(true);
con.setRequestMethod("POST");
con.setRequestProperty("connection", "keep-alive");
//This sets the UseCaches variable to false
con.setUseCaches(false);
String test = "<html><body><h1>Hello</h1></body></html>";
byte[] bytes = test.getBytes();
con.setRequestProperty("Content-length", String.valueOf(bytes.length));
con.setRequestProperty("Content-type", "text/html");
con.setRequestProperty("User-Agent", " Java1.3.1_04 ");
con.setRequestProperty("Accept", "text/html");
```

```
// Create the output streams
OutputStream out = con.getOutputStream();
// blank line separating header & body
out.println();
// send body
out.write(bytes);
out.flush();
// Create the input streams
BufferedReader in = new BufferedReader(new
InputStreamReader(con.getInputStream()));
String temp;
while((temp = in.readLine()) != null)
System.out.println(temp);
out.close();
in.close();
con.disconnect();
} catch(Exception e) {
e.printStackTrace();
System.exit(1);}}}
```

```
public class Server {
public static void main(String[] args) {
try {
ServerSocket serverSocket = new ServerSocket(12001);
System.out.println("HTTP Server (only POST implemented) is ready
and is listening on Port Number 12001 \n");
while(true) {
Socket clientSocket = serverSocket.accept();
System.out.println(clientSocket.getInetAddress().toString() + " " +
clientSocket.getPort());
// Create the input and output streams
BufferedReader in = new BufferedReader(new
InputStreamReader(clientSocket.getInputStream()));
OutputStream out = clientSocket.getOutputStream();
```

```
// Read the request message and display on console.
String temp;
while((temp=in.readLine()) != null)
System.out.println(temp);
String response = "HTTP/1.1 200 OK\n\r";
response = response + "Date: Fri, 04 May 2021 20:08:11 GMT\n\r";
response = response + "Server: Sanjits Server\n\r";
response = response + "Connection: close\n\r";
byte[] bytes = response.getBytes();
out.write(bytes);
out.flush();
in.close();
out.close();
}
} catch(Exception e) {
System.out.println("ERROR: " + e.getMessage());
System.exit(1);
}}}
```

```
********* The output on the server side is as follows ***************
HTTP Server (only POST implemented) is ready and is listening on Port
Number 12001
127.0.0.1/127.0.0.1 12001
POST / HTTP/1.1
Host: localhost:12001
Connection: keep-alive
Content-length: 18
Content-type: text/html
User-Agent: Java1.3.1_04
Accept: text/html

<html><body><h1>Hello</h1></body></html>
************************
```

EXAMPLE:#2 THE HTTPGET1.JAVA PROGRAM

```
import java.io.*;
import java.net.*;
public class HTTPGet1 {
// HTTP GET request
public static void getHTML(String website) throws Exception {
URL url = new URL(website);
HttpURLConnection conn = (HttpURLConnection) url.
openConnection();
conn.setRequestMethod("GET");
conn.setRequestProperty("User-Agent", "Chrome/51.0.2704.63");
conn.setRequestProperty("Accept-Language", "en-US,en");
int responseCode = conn.getResponseCode();
System.out.println("\nSending 'GET' request to URL : " +website);
System.out.println("Response Code : " + responseCode);
```

```
BufferedReader in = new BufferedReader(
new InputStreamReader(conn.getInputStream()));
StringBuilder response = new StringBuilder();
String line;
while ((line = in.readLine()) != null) {
response.append(line); }
in.close();
System.out.println( response.toString());
}
```

```
public static void main(String[] args) throws Exception
{
getHTML("https://docs.oracle.com/javase/tutorial/"); }
}
```

EXAMPLE:#3 THE HTTPPOST1.JAVA PROGRAM

```
import java.io.*;
import java.net.*;
public class HTTPPost1 {
// HTTP POST request
private static void postHTML(String website, String urlParameters)
throws Exception {
URL url = new URL(website);
HttpURLConnection conn = (HttpURLConnection) url.
openConnection();
conn.setRequestMethod("POST");
conn.setRequestProperty("User-Agent", "Chrome/51.0.2704.63");
conn.setRequestProperty("Accept-Language", "en-US,en");
```

```
// Send post request
conn.setDoOutput(true);
DataOutputStream OUT = new DataOutputStream(conn.
getOutputStream());
OUT.writeBytes(urlParameters);
OUT.flush();
OUT.close();
int responseCode = conn.getResponseCode();
System.out.println("\nSending 'POST' request to URL : " +
website);
System.out.println("Post parameters : " + urlParameters);
System.out.println("Response Code : " + responseCode);
```

```
BufferedReader in = new BufferedReader(
new InputStreamReader(conn.getInputStream()));
String inputLine;
StringBuffer response = new StringBuffer();
while ((inputLine = in.readLine()) != null) {
response.append(inputLine);
}
in.close();
//print result (first 1000 characters)
System.out.println(response.toString().substring(0,1000));
}
```

```
public static void main(String[] args) throws Exception
{
//Search java in Google
String website="https://www.google.co.uk/search?";
String urlParameters=""q=java";

postHTML(website, urlParameters);
}
}
```

---

6. Secure HTTP Requests (HTTPS) 🔒

- **Problem:** The original HTTP protocol is completely **unencrypted** (transmitted in plain text), presenting a security risk to intercepted packets.
- **Solution: HTTPS** (HyperText Transfer Protocol **Secure**) is the primary mechanism that provides HTTP with security.

HTTP vs. HTTPS Comparison Table

Property	**HTTP**	**HTTPS**
Name	HyperText Transfer Protocol	HyperText Transfer Protocol **Secure**
URL begins with	http://	https://
Port Number	80	443
Security	Insecure	Secure
Encryption	Absent	Present
Cryptographic Protocols	Not supported	Supported by **SSL** (Secure Sockets Layer)
Speed	Faster	Slower

```
try
{
URL u = new URL("https://www.httprecipes.com/1/5/https.php");
HttpsURLConnection http = u.openConnection();
// Now do something with the connection.
}
catch(MalformedURLException e)
{
System.out.println("Invalid URL");
}
catch(IOException e)
{
System.out.println("Error connecting: " + e.getMessage() );
}
```